

Pointer Sentinel Mixture Models

Stephen Merity
Caiming Xiong
James Bradbury
Richard Socher

MetaMind - A Salesforce Company, Palo Alto, CA, USA

SMERITY@SALESFORCE.COM
CXIONG@SALESFORCE.COM
JAMES.BRADBURY@SALESFORCE.COM
RSOCHER@SALESFORCE.COM

Abstract

Recent neural network sequence models with softmax classifiers have achieved their best language modeling performance only with very large hidden states and large vocabularies. Even then they struggle to predict rare or unseen words even if the context makes the prediction unambiguous. We introduce the pointer sentinel mixture architecture for neural sequence models which has the ability to either reproduce a word from the recent context or produce a word from a standard softmax classifier. Our pointer sentinel-LSTM model achieves state of the art language modeling performance on the Penn Treebank (70.9 perplexity) while using far fewer parameters than a standard softmax LSTM. In order to evaluate how well language models can exploit longer contexts and deal with more realistic vocabularies and larger corpora we also introduce the freely available WikiText corpus.¹

1. Introduction

A major difficulty in language modeling is learning when to predict specific words from the immediate context. For instance, imagine a new person is introduced and two paragraphs later the context would allow one to very accurately predict this person's name as the next word. For standard neural sequence models to predict this name, they would have to encode the name, store it for many time steps in their hidden state, and then decode it when appropriate. As the hidden state is limited in capacity and the optimization of such models suffer from the vanishing gradient problem, this is a lossy operation when performed over many timesteps. This is especially true for rare words.

Models with soft attention or memory components have been proposed to help deal with this challenge, aiming to allow for the retrieval and use of relevant previous hidden

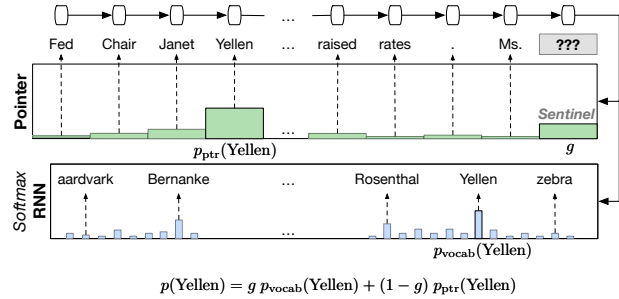


Figure 1. Illustration of the pointer sentinel-RNN mixture model. g is the mixture gate which uses the sentinel to dictate how much probability mass to give to the vocabulary.

states, in effect increasing hidden state capacity and providing a path for gradients not tied to timesteps. Even with attention, the standard softmax classifier that is being used in these models often struggles to correctly predict rare or previously unknown words.

Pointer networks (Vinyals et al., 2015) provide one potential solution for rare and out of vocabulary (OoV) words as a pointer network uses attention to select an element from the input as output. This allows it to produce previously unseen input tokens. While pointer networks improve performance on rare words and long-term dependencies they are unable to select words that do not exist in the input.

We introduce a mixture model, illustrated in Fig. 1, that combines the advantages of standard softmax classifiers with those of a pointer component for effective and efficient language modeling. Rather than relying on the RNN hidden state to decide when to use the pointer, as in the recent work of Gülçehre et al. (2016), we allow the pointer component itself to decide when to use the softmax vocabulary through a sentinel. The model improves the state of the art perplexity on the Penn Treebank. Since this commonly used dataset is small and no other freely available alternative exists that allows for learning long range dependencies, we also introduce a new benchmark dataset for language modeling called WikiText.

¹Available for download at the [WikiText dataset site](#)

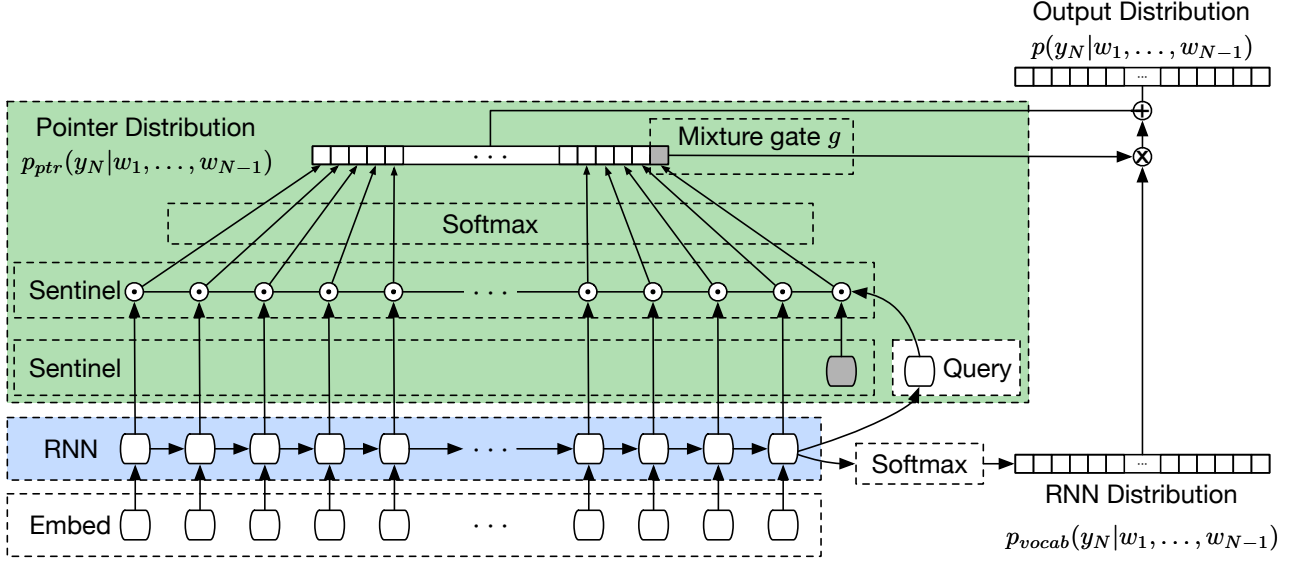


Figure 2. Visualization of the pointer sentinel-RNN mixture model. The query, produced from applying an MLP to the last output of the RNN, is used by the pointer network to identify likely matching words from the past. The $\odot$ nodes are inner products between the query and the RNN hidden states. If the pointer component is not confident, probability mass can be directed to the RNN by increasing the value of the mixture gate g via the sentinel, seen in grey. If $g = 1$ then only the RNN is used. If $g = 0$ then only the pointer is used.

2. The Pointer Sentinel for Language Modeling

Given a sequence of words $w_1, \dots, w_{N-1}$, our task is to predict the next word w_N .

2.1. The softmax-RNN Component

Recurrent neural networks (RNNs) have seen widespread use for language modeling (Mikolov et al., 2010) due to their ability to, at least in theory, retain long term dependencies. RNNs employ the chain rule to factorize the joint probabilities over a sequence of tokens: $p(w_1, \dots, w_N) = \prod_{i=1}^N p(w_i | w_1, \dots, w_{i-1})$. More precisely, at each time step i , we compute the RNN hidden state h_i according to the previous hidden state h_{i-1} and the input x_i such that $h_i = \text{RNN}(x_i, h_{i-1})$. When all the $N - 1$ words have been processed by the RNN, the final state h_{N-1} is fed into a softmax layer which computes the probability over a vocabulary of possible words:

$$p_{\text{vocab}}(w) = \text{softmax}(U h_{N-1}), \quad (1)$$

where $p_{\text{vocab}} \in \mathbb{R}^V$, $U \in \mathbb{R}^{V \times H}$, H is the hidden size, and V the vocabulary size. RNNs can suffer from the vanishing gradient problem. The LSTM (Hochreiter & Schmidhuber, 1997) architecture has been proposed to deal with this by updating the hidden state according to a set of gates. Our work focuses on the LSTM but can be applied to any RNN architecture that ends in a vocabulary softmax.

2.2. The Pointer Network Component

In this section, we propose a modification to pointer networks for language modeling. To predict the next word in the sequence, a pointer network would select the member of the input sequence $p(w_1, \dots, w_{N-1})$ with the maximal attention score as the output.

The simplest way to compute an attention score for a specific hidden state is an inner product with all the past hidden states h_i , with each hidden state $h_i \in \mathbb{R}^H$. However, if we want to compute such a score for the most recent word (since this word may be repeated), we need to include the last hidden state itself in this inner product. Taking the inner product of a vector with itself results in the vector's magnitude squared, meaning the attention scores would be strongly biased towards the most recent word. Hence we project the current hidden state to a query vector q first. To produce the query q we compute

$$q = \tanh(W h_{N-1} + b), \quad (2)$$

where $W \in \mathbb{R}^{H \times H}$, $b \in \mathbb{R}^H$, and $q \in \mathbb{R}^H$. To generate the pointer attention scores, we compute the match between the previous RNN output states h_i and the query q by taking the inner product, followed by a softmax activation function to obtain a probability distribution:

$$z_i = q^T h_i, \quad (3)$$

$$a = \text{softmax}(z), \quad (4)$$

where $z \in \mathbb{R}^L$, $a \in \mathbb{R}^L$, and L is the total number of hidden

states. The probability mass assigned to a given word is the sum of the probability mass given to all token positions where the given word appears:

$$p_{\text{ptr}}(w) = \sum_{i \in I(w, x)} a_i, \quad (5)$$

where $I(w, x)$ results in all positions of the word w in the input x and $p_{\text{ptr}} \in \mathbb{R}^V$. This technique, referred to as pointer sum attention, has been used for question answering (Kadlec et al., 2016).

Given the length of the documents used in language modeling, it may not be feasible for the pointer network to evaluate an attention score for all the words back to the beginning of the dataset. Instead, we may elect to maintain only a window of the L most recent words for the pointer to match against. The length L of the window is a hyperparameter that can be tuned on a held out dataset or by empirically analyzing how frequently a word at position t appears within the last L words.

To illustrate the advantages of this approach, consider a long article featuring two sentences *President Obama discussed the economy* and *President Obama then flew to Prague*. If the query was *Which President is the article about?*, probability mass could be applied to *Obama* in either sentence. If the question was instead *Who flew to Prague?*, only the latter occurrence of *Obama* provides the proper context. The attention sum model ensures that, as long as the entire attention probability mass is distributed on the occurrences of *Obama*, the pointer network can achieve zero loss. This flexibility provides supervision without forcing the model to put mass on supervision signals that may be incorrect or lack proper context. This feature becomes an important component in the pointer sentinel mixture model.

2.3. The Pointer Sentinel Mixture Model

While pointer networks have proven to be effective, they cannot predict output words that are not present in the input, a common scenario in language modeling. We propose to resolve this by using a mixture model that combines a standard softmax with a pointer.

Our mixture model has two base distributions: the softmax vocabulary of the RNN output and the positional vocabulary of the pointer model. We refer to these as the RNN component and the pointer component respectively. To combine the two base distributions, we use a gating function $g = p(z_i = k | x_i)$ where z_i is the latent variable stating which base distribution the data point belongs to. As we only have two base distributions, g can produce a scalar in the range $[0, 1]$. A value of 0 implies that only the pointer

is used and 1 means only the softmax-RNN is used.

$$p(y_i | x_i) = g p_{\text{vocab}}(y_i | x_i) + (1 - g) p_{\text{ptr}}(y_i | x_i). \quad (6)$$

While the models could be entirely separate, we re-use many of the parameters for the softmax-RNN and pointer components. This sharing minimizes the total number of parameters in the model and capitalizes on the pointer network’s supervision for the RNN component.

2.4. Details of the Gating Function

To compute the new pointer sentinel gate g , we modify the pointer component. In particular, we add an additional element to z , the vector of attention scores as defined in Eq. 3. This element is computed using an inner product between the query and the sentinel² vector $s \in \mathbb{R}^H$. This change can be summarized by changing Eq. 4 to:

$$a = \text{softmax}([z; q^T s]). \quad (7)$$

We define $a \in \mathbb{R}^{V+1}$ to be the attention distribution over both the words in the pointer window as well as the sentinel state. We interpret the last element of this vector to be the gate value: $g = a[V + 1]$.

Any probability mass assigned to g is given to the standard softmax vocabulary of the RNN. The final updated, normalized pointer probability over the vocabulary in the window then becomes:

$$p_{\text{ptr}}(y_i | x_i) = \frac{1}{1 - g} a[1 : V], \quad (8)$$

where we denoted $[1 : V]$ to mean the first V elements of the vector. The final mixture model is the same as Eq. 6 but with the updated Eq. 8 for the pointer probability.

This setup encourages the model to have both components compete: use pointers whenever possible and back-off to the standard softmax otherwise. This competition, in particular, was crucial to obtain our best model. By integrating the gating function directly into the pointer computation, it is influenced by both the RNN hidden state and the pointer window’s hidden states.

2.5. Motivation for the Sentinel as Gating Function

To make the best decision possible regarding which component to use the gating function must have as much context as possible. As we increase both the number of timesteps and the window of words for the pointer component to consider, the RNN hidden state by itself isn’t guaranteed to

²A sentinel value is inserted at the end of a search space in order to ensure a search algorithm terminates if no matching item is found. Our sentinel value terminates the pointer search space and distributes the rest of the probability mass to the RNN vocabulary.